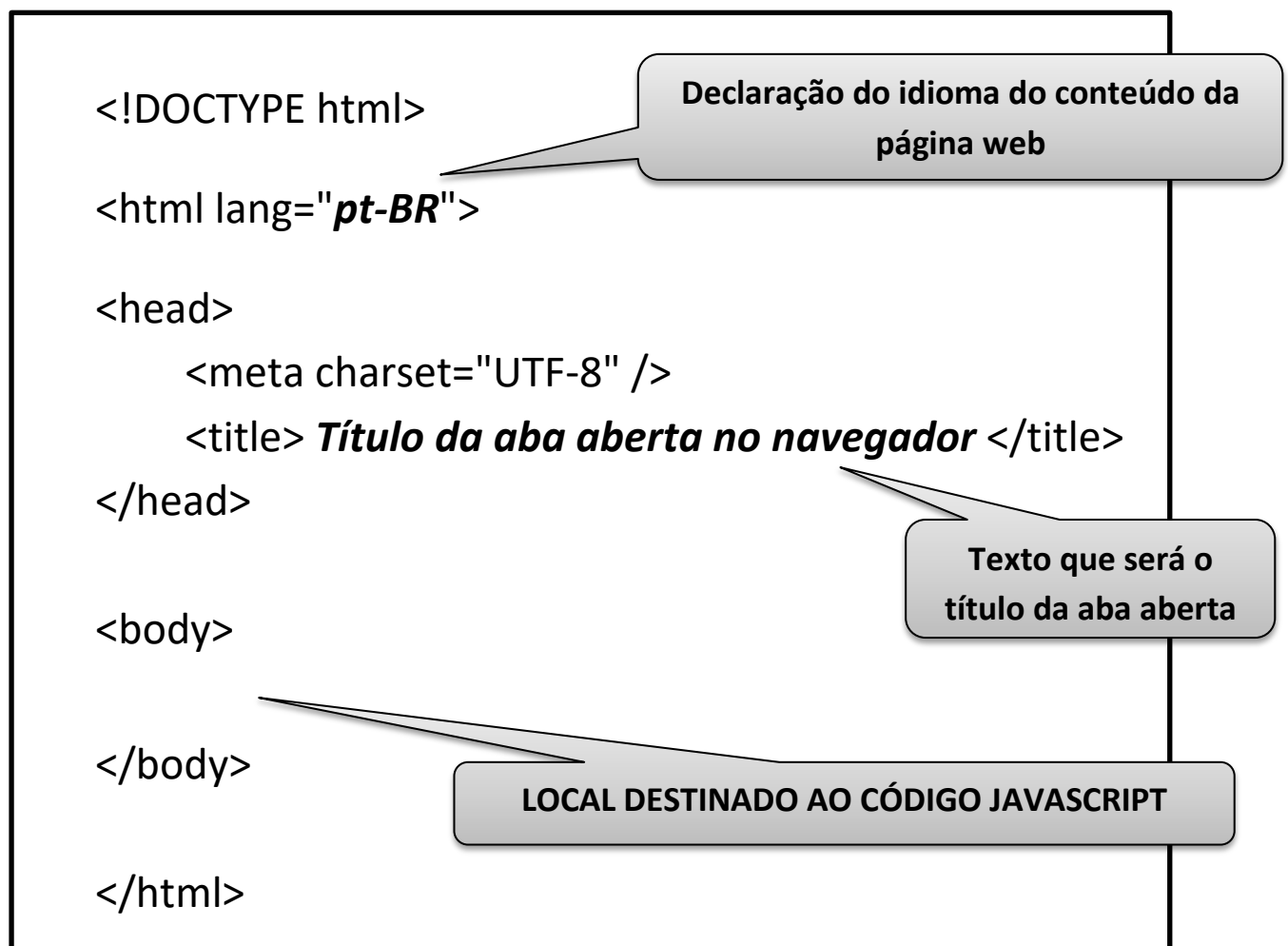


Criando algoritmos na prática utilizando linguagem JavaScript (JS)

Para criar algoritmos usando a linguagem JavaScript primeiro é necessário construir o esqueleto de uma página HTML. O motivo disso é simples: que essa linguagem é executada dentro de um arquivo HTML.

O código JavaScript deve ser inserido entre as tags <body> e </body>.

Esqueleto básico da página HTML:



Todo código JS deverá estar entre as tags <script> e </script>.

```
<!DOCTYPE html>

<html lang="pt-BR">

<head>
  <meta charset="UTF-8" />
  <title> Título da aba aberta no navegador </title>
</head>

<body>

  <script type="text/javascript">

    </script>

</body>

</html>
```

Todo o código JavaScript deverá ser colocado entre as tags <script> e </script>

OBS.1: Até a versão 4 da linguagem HTML era obrigatório inserir o tipo de script na tag de abertura: type="text/javascript". A partir da versão 5 (atual) isso passou a ser opcional pois, em sua ausência, adota-se como padrão que se trata de um script da linguagem JS.

OBS.2: Caso haja interesse em adicionar códigos HTML na página essa ação poderia ocorrer normalmente antes ou depois do código JS.

OBS.3: Todo comando em JS deverá finalizar com ponto e vírgula (;).

OBS.4: Atenção - existe diferença entre letras maiúsculas e minúsculas na linguagem JS, por isso é necessário cuidado ao digitar comandos e nomes de variáveis e constantes.

Conversão de pseudocódigo para linguagem JavaScript

1 - Criação de variável:

```
var nome da variável;  
var nome da variável = conteúdo;
```

Exemplos: **var** contador;
 var contador = 1;

2 - Criação de constante:

```
const nome da constante = conteúdo;
```

Exemplo: **const** numero = 100;

OBS.5: Uma constante não pode ter seu conteúdo alterado após sua criação, dessa forma, ao ser criada já deverá ter seu conteúdo definido.

3 - Exibir um conteúdo:

```
document.write(contúdo);
```

Exemplo: **document.write**("conteúdo");
 document.write(nome da variável);

4 - Estrutura de decisão SE ... SENÃO ... FIM SE:

Corresponde ao "SE"

Corresponde ao "SENÃO"

```
if (condição) {  
    outros comandos }  
else {  
    outros comandos  
};
```

4 - Estrutura de decisão SE ... SENÃO SE ... SENÃO ... FIM SE:

Corresponde ao "SE"

Corresponde ao "SENÃO SE"

Corresponde ao "SENÃO"

```
if (condição) {  
    outros comandos }  
else if (condição) {  
    outros comandos }  
else {  
    outros comandos  
};
```

5 – Operadores de comparação:

Operador	Descrição	Exemplo
Igual (==)	Retorna verdadeiro caso os operandos sejam iguais.	2 == 2 'Teste' == 'Teste' Var1 == var2
Não igual (!=)	Retorna verdadeiro caso os operandos não sejam iguais.	var1 != 4 var2 != "abc"
Maior que (>)	Retorna verdadeiro caso o operando da esquerda seja maior que o da direita.	var2 > var1
Maior que ou igual (>=)	Retorna verdadeiro caso o operando da esquerda seja maior ou igual ao da direita.	var2 >= var1
Menor que (<)	Retorna verdadeiro caso o operando da esquerda seja menor que o da direita.	var1 < var2
Menor que ou igual (<=)	Retorna verdadeiro caso o operando da esquerda seja menor ou igual ao da direita.	var1 <= var2

6 – Operadores de atribuição:

Nome	Operador encurtado	Significado
Atribuição	x = y	x = y
Atribuição de adição	x += y	x = x + y
Atribuição de subtração	x -= y	x = x - y
Atribuição de multiplicação	x *= y	x = x * y
Atribuição de divisão	x /= y	x = x / y
Atribuição de resto	x %= y	x = x % y
Atribuição exponencial	x **= y	x = x ** y

7 – Operadores lógicos:

Operador	Utilização	Descrição
E lógico (&&)	expr1 && expr2	Retorna verdadeiro quando todas as condições forem verdadeiras. Se houver uma condição falsa retornará falso .
OU lógico ()	expr1 expr2	Retorna falso quando todas as condições forem falsas. Se houver uma condição verdadeira retornará verdadeiro .
NOT lógico (!)	!expr	Realiza a negação de um resultado, ou seja, inverte o retorno. Se o retorno é verdadeiro então se tornará falso . Caso o retorno seja falso se tornará verdadeiro .

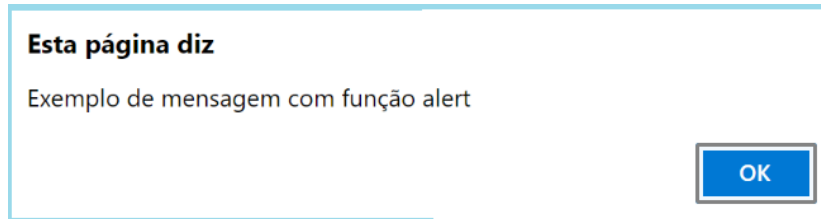
8 – Exibir um conteúdo em forma de popup:

`alert(conteúdo);`

Exemplo: `alert("conteúdo");`
`alert(nome da variável);`

O alert é uma das mais simples caixas de diálogo, com uma aparência simples e intuitiva. Sua principal função é mostrar ao usuário uma mensagem e um botão de confirmação de que o usuário tenha visto a mensagem.

`alert("Exemplo de mensagem com função alert");`



OBS.1: Existe também a possibilidade de pular linha na mensagem exibida pela função alert(), para isso basta inserir “\n” no local desejado para quebra de linha. Exemplo: `alert("Exemplo de mensagem \ncom função alert")`.

OBS.2: É possível concatenar (“somar”) conteúdos distintos usando o símbolo “+”.

9 – Receber um conteúdo do usuário:

`variável = prompt(conteúdo);`

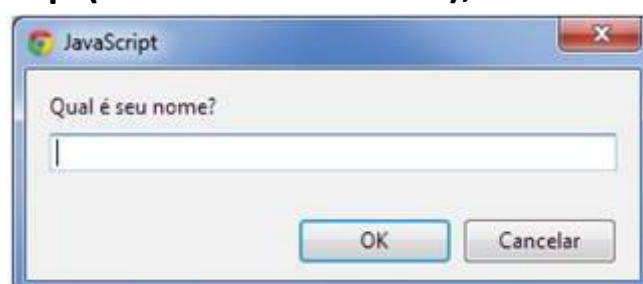
A função **`prompt()`** abre uma janela popup com uma caixa de texto para coletar informações do usuário. O retorno dessa função é o texto digitado. Esse retorno geralmente é armazenado em uma variável.

A estrutura básica dessa caixa de diálogo é:

- Um local para digitação da informação
- Botão OK
- Botão Cancelar

Exemplo:

```
var nome;  
nome = prompt("Qual é seu nome?");
```



10 – Converter um conteúdo string para número:

```
variável = Number(conteúdo);
```

A função **Number()** converte um conteúdo string para número desde que esse conteúdo string seja formado por caracteres que possam ser convertidos para número.

Exemplo:

```
var numero = "232";  
num = Number(numero);
```

11 – Converter um conteúdo para número inteiro:

```
variável = parseInt(conteúdo);
```

A função **parseInt()** converte um conteúdo string para número inteiro (sem casas decimais) desde que esse conteúdo string seja formado por caracteres que possam ser convertidos para número inteiro.

Exemplo:

```
var numero = "232";  
num = parseInt(numero);
```

12 – Converter um conteúdo para número com casas decimais:

```
variável = parseFloat(conteúdo);
```

A função **parseFloat()** converte um conteúdo string para número com casas decimais desde que esse conteúdo string seja formado por caracteres que possam ser convertidos para número.

Exemplo:

```
var numero = "232.90";  
num = parseFloat(numero);
```

13 – Limitar a quantidade de casas decimais:

variável = variável.**toFixed**(quantidade de casas decimais);

A função **toFixed()** limita a quantidade de casas decimais de um número ou variável numérica. A quantidade expressada entre parênteses será aplicada no número ou variável numérica. Quando a casa decimal imediatamente subsequente ao limite expressado for maior que 5 ocorrerá um arredondamento “para cima”.

Exemplo 1:

```
var numero = 78.56139;
```

```
numero = numero.toFixed(2);
```

Resultado será 78.56

Exemplo 2:

```
var numero = 5.56139;
```

```
numero = numero.toFixed(1);
```

Resultado será 5.6

Exemplo 3:

```
var numero = 167.91876;
```

```
numero = numero.toFixed(2);
```

Resultado será 167.92

OBS.1: Quando a função **toFixed** for utilizada em variáveis ela somente funcionará quando o conteúdo for numérico com casas decimais. Recomenda-se inicialmente validar se o conteúdo existente na variável é este (numérico com casas decimais) antes de aplicar a função referida.

OBS.2: A função que converte um conteúdo para numérico com casas decimais é **parseFloat**.

14 – Verificar se um conteúdo não é numérico

```
variável = isNaN(conteúdo);
```

A função `isNaN()` é usada para verificar se um parâmetro não é numérico.

NaN significa **Not-A-Number**.

Se o conteúdo utilizado nessa função não for numérico retornará **true** (ou 1), caso contrário retornará **false**.

Exemplo:

```
var num=5;
if (isNaN(num)) {
    alert("Não é um número");
}
else {
    alert("É um número");
};
```

15 – Converter um conteúdo para string:

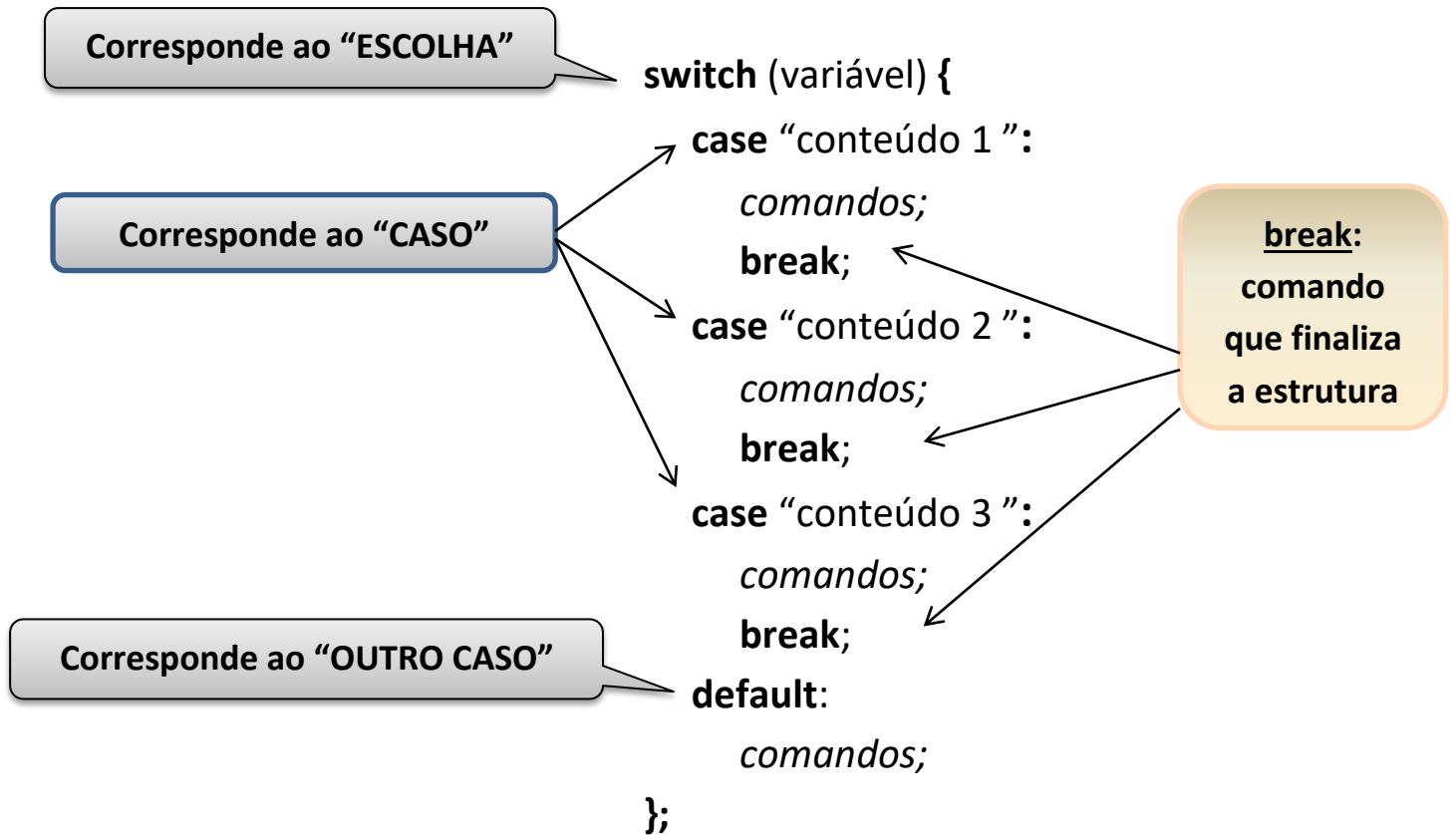
```
variável = conteúdo.toString();
```

A função **`toString()`** converte um conteúdo para string.

Exemplo:

```
var conteudo = 123;
conversao = conteudo.toString();
```


16 – Estrutura de decisão ESCOLHA ... CASO... OUTRO CASO :



OBS.: Ao final de cada item "case" é importante inserir o comando break para finalizar essa estrutura de decisão, caso contrário os demais comandos dos outros itens também serão executados.

17 – Estrutura de repetição ENQUANTO ... FAÇA :

Corresponde ao “ENQUANTO”

```
while (condição) {  
    comandos;  
};
```

18 – Estrutura de repetição FAÇA ... ENQUANTO :

Corresponde ao “FAÇA”

```
do (condição) {  
    comandos;
```

Corresponde ao “ENQUANTO”

```
}  
while (condição);
```

19 – Estrutura de repetição PARA ... ATÉ ... SEGUINTE :

```
for (variável; condição; alteração da variável) {  
    comandos;  
};
```

OBS.: Recordando que toda estrutura de repetição precisa de um contador para controlar a “condição” utilizada.